

BALANCED TREE – B-TREE

Yapisi · Minimum/Maksimum Derece · Ekleme & Split · Yukseklik Analizi · Karmasiklik Turetme

Ne için B-Tree? Disk tabanlı veri yapıları için tasarlandı. Her düğüm $t - 1$ ile $2t - 1$ arasında anahtar sigdırılır – tek disk okumada çok anahtar. BST’de n eleman için $O(\log n)$ disk okuma gerekirken, B-Tree’de her düğüm yüzlerce anahtar tutabilir, derinlik çok küçük kalır.

1. B-Tree Yapisi ve Ozellikleri

B-Tree Yapisi ve Ozellikleri

Temel ozellikler:

- Her düğüm birden fazla anahtar barindirabilir.
- Tüm yapraklar aynı seviyededir (tam dengeli).
- Tüm veri sıralı tutulur – inorder = sıralı liste.
- Arama, ekleme, silme: $O(\log n)$.

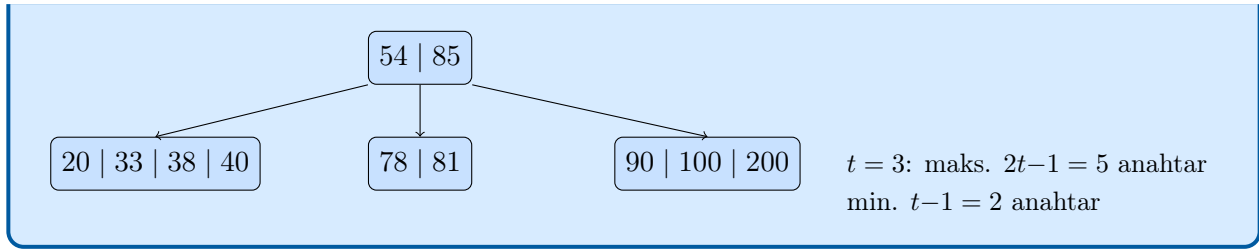
B-Tree derecesi: $t \geq 2$ (minimum degree)

Ozellik	t (minimum degree)	t (maksimum degree)
t neyi ifade eder?	minimum çocuk sayısı	maksimum çocuk sayısı
maksimum anahtar	$2t - 1$	$t - 1$
minimum anahtar	$t - 1$	$\lceil t/2 \rceil - 1$
maksimum çocuk	$2t$	t
minimum çocuk	t	$\lceil t/2 \rceil$

Neden bu sinirlar?

- **Maksimum $2t - 1$ anahtar:** Her disk bloğu belirli bir boyut. Blok dolunca split yapılır. $2t - 1$ anahtarı olan düğüm ortadan bölünür, orta eleman yukarıya çıkar – sol ve sağ her biri $t - 1$ anahtarla kalır.
- **Minimum $t - 1$ anahtar (kok haric):** Merge onlemek için düğümler en az yaridan dolu olmalı. Aksi halde ağac bosu boğunca büyür.
- **Minimum t çocuk:** $t - 1$ anahtar, t aralık oluşturur.

Gorsel – $t = 3$ B-Tree ornegi:



Pratik – Yapi

(a) $t = 2$ (2-3-4 agaci) icin: maksimum anahtar, minimum anahtar, maksimum ve minimum cocuk sayisi nedir? _____

(b) Yukaridaki ornekte neden 54 ve 85 kokte, diger anahtarlar yapraklarda? BST ozelligini kontrol edin: $20 < 33 < 38 < 40 < 54 < 78 < 81 < 85 < 90 < 100 < 200$ ____

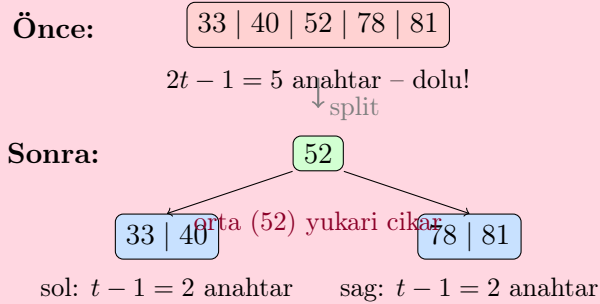
2. Ekleme ve Bolunme (Split)

B-Tree'ye ekleme her zaman yapraga yapilir. Yaprak dolu ($2t-1$ anahtar) ise bolunme (split) tetiklenir. Split yukariya dogru yayilabilir.

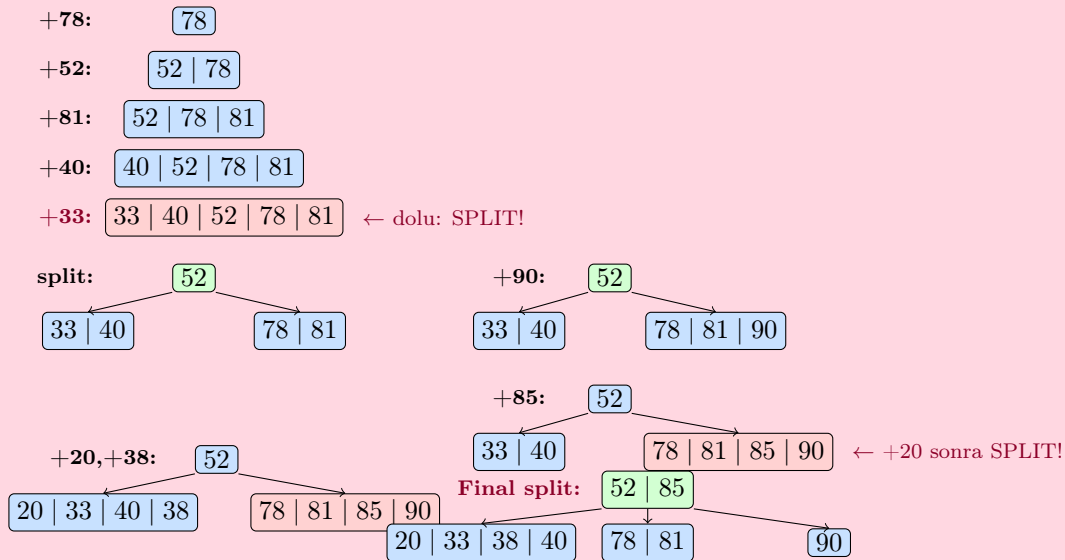
Ekleme ve Bolunme (Split)

Ekleme kurali: Kokte basla, dogru yapraga in. Yolda dolu dugum gorursen ONCE bol, sonra in. Boylece split geri yukari yayilmaz.

Split nasıl çalışır? ($t = 3$, $2t - 1 = 5$ dolu dugum bolunuyor)



Notlardaki örnek – adım adım ekleme ($t = 3$): **Siralama:**
 78, 52, 81, 40, 33, 90, 85, 20, 38



Split karmasikligi: Bir split $O(t)$ zaman (orta eleman kopyalama + cocuklar bolunur). Yol boyunca en fazla $O(h)$ split olabilir. Toplam ekleme: $O(t \cdot h)$.

Pratik – Ekleme

(a) $t = 2$ B-Tree'ye $\{10, 20, 5, 6, 12, 30, 7, 17\}$ sirasinda ekleyin. Her split adimini gosterin.

(b) $t = 3$, 5 dolu dugume 38 ekleniyor. Split sonrasi ag nasil gorunur? _____

3. Yukseklik ve Karmasiklik Analizi

Yukseklik ve Karmasiklik Analizi

Adim 1 – Minimum dugum sayisi: B-Tree ne kadar dolu olabilir?

En az n anahtar tutan B-Tree'nin yuksekligi en fazla ne kadar olur? Bunun icin **minimum** anahtar sayisini bulmak gerekir – yani agac ne kadar bos olabilir?

- Kok: en az 1 anahtar (2 cocuk)
- Diger her dugum: en az $t - 1$ anahtar (t cocuk)
- Derinlik d olan agacta: seviye 0'da 2 cocuk, her seviyede t katlanir

Seviye i 'de minimum dugum sayisi:

$$\text{seviye 0 (kok):} 1 \quad \text{seviye 1:} 2 \quad \text{seviye 2:} 2t \quad \text{seviye } i: 2t^{i-1} \quad (i \geq 1)$$

Adim 2 – Minimum toplam anahtar sayisi formulu

Her dugumde en az $t - 1$ anahtar. Yukseklik d olan agactaki minimum toplam anahtar:

$$n_{\min} = \underbrace{1}_{\text{kok}} + \underbrace{(t-1) \cdot 2}_{\text{sev.1}} + \underbrace{(t-1) \cdot 2t}_{\text{sev.2}} + \cdots + \underbrace{(t-1) \cdot 2t^{d-1}}_{\text{sev.}d} = 1 + (t-1) \cdot 2 \sum_{i=0}^{d-1} t^i$$

Geometrik seri: $\sum_{i=0}^{d-1} t^i = \frac{t^d - 1}{t - 1}$

$$n_{\min} = 1 + (t-1) \cdot 2 \cdot \frac{t^d - 1}{t - 1} = 1 + 2(t^d - 1) = 2t^d - 1$$

Yani n anahtarli bir B-Tree icin:

$$n \geq n_{\min} = 2t^d - 1$$

Adim 3 – Yukseklik ustune alt sinir: $d \leq \log_t \frac{n+1}{2}$

$n \geq 2t^d - 1$ ifadesini d icin cozelim:

$$n \geq 2t^d - 1 \Rightarrow n + 1 \geq 2t^d \Rightarrow t^d \leq \frac{n+1}{2} \Rightarrow d \leq \log_t \frac{n+1}{2}$$

Anlami: Derinlik en fazla $\log_t \frac{n+1}{2}$. t buyukse (her dugumde cok anahtar) derinlik cok kucuk kalir. Ornegin $t = 1000$: $n = 10^9$ eleman icin $d \leq 3$.

$$d = O(\log_t n)$$

Adim 4 – Ekleme zamani: tam turetme

Ekleme iki asama:

1. **Yapraga in:** Her seviyede dugum icinde ikili arama yap: $O(\log t)$. d seviye: toplam $O(h \cdot \log t)$.
2. **Split (bolunme):** Her seviyede en fazla 1 split, her split $O(t)$. d seviye: toplam $O(h \cdot t)$.

Toplam ekleme zamani:

$$T(n) = O(t \cdot h + h) = O(h(t + 1)) = O(h \cdot t)$$

$h = O(\log_t n)$ yazarak:

$$T(n) = O(t \cdot \log_t n) = O\left(t \cdot \frac{\log n}{\log t}\right)$$

Notlardaki daha acik yazi ile:

$$T(n) = \left(t \cdot \left(\log_t^{(n+1)} - 1\right)\right) + \left(\log_t^{(n+1)} - 1\right) = O\left(\log_t^{(n+1)}\right) \cdot (t + 1)$$

Neden $O(\log n)$ deriz? Cunku t sabittir. Sabit t icin:

$$O(t \log_t n) = O\left(\frac{t}{\log t}\right) \cdot O(\log n) = O(\log n)$$

Buyuk-O'da sabit katsayilar duser: $T(n) = O(\log n)$

Adim 5 – $t = 2$ ozel durum: $n = 2^{d+1} - 1$

Notlarda ayrica $t = 2$ icin ozel durum var. $t = 2$: her dugumde min 1, maks 3 anahtar (2-3-4 agaci).

$$n_{\min} = 2 \cdot 2^d - 1 = 2^{d+1} - 1 \Rightarrow d = \log_2(n + 1) - 1$$

Yani $n + 1$ elemanli tam ikili agac gibi davranir.

Ozet tablo:

Islem	Disk okuma	CPU	Not
Arama	$O(\log_t n)$	$O(t \log_t n)$	Her dugumde ikili arama
Ekleme	$O(\log_t n)$	$O(t \log_t n)$	Split dahil
Silme	$O(\log_t n)$	$O(t \log_t n)$	Merge/borrow dahil
Sabit t icin tumu $O(\log n)$. Buyuk t disk okumalari azaltir.			

Pratik – Analiz

(a) $t = 10$, $n = 10^6$ eleman. Maksimum yukseklik nedir? $d \leq \log_{10}(500000)$ hesaplayin.

(b) Neden BST degil de B-Tree kullanilir? Disk modeline gore aciklayin. _____

(c) Notlardaki turetmede minimum dugum sayisi toplami: $(t-1) \sum_{i=0}^d t^i$ ifadesini acin ve neden $2t^d - 1$ oldugunu gosterin. _____

(d) $T(n) = O(t \cdot \log_t n)$ ifadesinde t 'yi optimize etmek (en kucuk T 'yi bulmak) icin t 'ye gore turevi alin. $t = e$ oldugunda minimum. Bu disk sistemlerinde t 'nin neden genellikle 100-1000 arasinda secildigini aciklar mi? (Ipucu: disk blok boyutu $\approx 4\text{KB}$, her anahtar ≈ 4 byte) _____

B-Tree vs BST vs Hash:

- **BST:** Her dugum 1 anahtar, disk icin verimsiz ($O(n)$ disk okuma mumkun).
- **B-Tree:** Her dugum $t-1$ ile $2t-1$ anahtar; disk blogu = dugum. t buyukse ($t = 100$) milyonlarca eleman 3-4 disk okumada bulunur.
- **Hash:** $O(1)$ ama sirasiz, aralik sorgusu yapamaz.
- B-Tree: veritabani indeksleri (MySQL InnoDB, PostgreSQL) icin altin standart.